

PROJECT 2: MODEL ANALYSIS PROJECT

- **Vision:** *Programming is more than writing code.* The ultimate goal of the projects in this course is that you learn to structure a programming project from start to finish. Your answers should of course be correct, but your code should also be well-structured and documented.
- **Objectives:** In your model analysis project, you should show that you can
 1. Apply numerical model analysis methods (optimization, root-finding, etc.)
 2. Present results in text form and in easy-to-understand figures
 3. Teach yourself new functionality in a package you already know
 4. Structure a code project
 5. Document code

- **Structure:** Your model analysis project should consist of
 1. A single self-contained notebook (.ipynb) presenting the analysis
 2. Documented Python module files (.py)

Starting point: Use the files `Consumer.py` and `Government.py`. Everything except the methods marked with `pass` is written for you.

Example of structure: See this [repository](#) and `ProgEcon-lectures/08_DataProject/01_Structure/`

- **Hand-in:** Upload in Absalon
- **Deadline:** See [Calendar](#).
- **Feedback:** Your TA will provide feedback on your project.
- **Exam:** Your model analysis project will be a part of your exam portfolio. You can incorporate feedback before handing in the final version.

This first version does not need to be perfect!

1 A consumer with two nests

A consumer buys three goods in quantities $x_1, x_2, x_3 \geq 0$ at prices $p_1, p_2, p_3 > 0$, and has income $I > 0$.

Goods 2 and 3 belong together in a **nest**, while good 1 is on its own. Say good 1 is **food**, and goods 2 and 3 are **bus trips** and **train trips**. Bus and train are both ways of travelling, so they are much more like each other than either of them is like food. That is all the nest says.

First the two travel goods are combined into a **travel composite**

$$x_B = [\beta x_2^{\rho_B} + (1 - \beta)x_3^{\rho_B}]^{\frac{1}{\rho_B}}, \quad (1)$$

and then food and travel are combined into utility,

$$u(x_1, x_2, x_3) = [\alpha x_1^{\rho_A} + (1 - \alpha)x_B^{\rho_A}]^{\frac{1}{\rho_A}}, \quad (2)$$

where $\alpha \in (0, 1)$ is the weight on food and $\beta \in (0, 1)$ is the weight on the bus.

The two exponents are $\rho_A = 1 - \frac{1}{\sigma_A}$ and $\rho_B = 1 - \frac{1}{\sigma_B}$, where $\sigma_A > 0$ and $\sigma_B > 0$ say how easily the consumer can **substitute** one good for the other:

- $\sigma_B < 1$: bus and train are **complements**. The consumer wants both, and cannot get far by replacing one with the other.
- $\sigma_B > 1$: bus and train are **substitutes**. Either one will do, so the consumer mostly buys whichever is cheaper.

and the same for σ_A , between food and travel. We always use $\sigma \neq 1$, so that $\rho \neq 0$ and you never divide by zero in the exponent.

1.1 The problem in nested budget shares

Rather than in quantities, we write the problem in **budget shares**, $s_j = \frac{p_j x_j}{I}$, so that $x_j = \frac{s_j I}{p_j}$. Utility is increasing in all three goods, so the consumer never throws money away and spends everything, $s_1 + s_2 + s_3 = 1$.

We now describe the choice in the **same two steps as the nests**:

1. $s_1 \in [0, 1]$: the share of income spent on **food**. That leaves the share $1 - s_1$ for travel.
2. $w \in [0, 1]$: the share of the **travel budget** spent on the **bus**.

The three budget shares then follow as

$$s_1, \quad s_2 = (1 - s_1) w, \quad s_3 = (1 - s_1) (1 - w), \quad (3)$$

and the quantities as $x_j = \frac{s_j I}{p_j}$. The problem is

$$V = \max_{s_1 \in [0,1], w \in [0,1]} u \left(\frac{s_1 I}{p_1}, \frac{(1 - s_1) w I}{p_2}, \frac{(1 - s_1) (1 - w) I}{p_3} \right). \quad (4)$$

We call (s_1, w) the **nested budget shares**.

This is the reparametrization from section 6.4 of the optimization lecture. Whatever (s_1, w) you pick, the three shares in eq. 3 sum to one, so the budget constraint holds *exactly*. The constraint set is therefore the box $[0, 1] \times [0, 1]$ -- and a box is precisely what L-BFGS-B takes as its bounds. No penalties and no constraint handling of any kind are needed anywhere in this project.

1.2 Calibration

Every question below should be answered for **two calibrations**, where σ_B differs:

Parameter	Symbol	Complements	Substitutes
Weight on food	α	0.60	0.60
Weight on the bus	β	0.50	0.50
Substitution, food vs. travel	σ_A	0.80	0.80
Substitution, bus vs. train	σ_B	0.40	3.00
Price of food	p_1	1.0	1.0
Price of a bus trip	p_2	1.0	1.0
Price of a train trip	p_3	1.5	1.5
Income	I	10.0	10.0

Hint: Write your code so that running everything again for the second calibration costs you *one extra line*, not a second copy of the code. In `Consumer.py` a consumer is created with `model = ConsumerClass(par={'sigma_B':3.0})`.

1.3 How do you know your answer is right?

There is **no formula** for the solution of this model, so you cannot look the answer up. Two cheap checks:

1. Is the answer **possible**? All three shares should be strictly between 0 and 1 and sum to one, and all three quantities should be positive.
2. Do **two different methods** agree? In section 2 you solve the same problem twice.

2 Solving the model numerically

We solve eq. 4 twice, in two very different ways.

2.1 A two-dimensional grid search

The idea is the one from the optimization lecture: lay a grid over everything the consumer can afford, compute utility in every point, and keep the best one. Nothing clever happens -- and that is the point.

1. Make a grid of N values of s_1 and N values of w , both from 0 to 1, compute utility in every combination, and keep the best point. Implement it as `solve_grid()`. Report both (s_1, w) and the three budget shares it implies.

Hint: `np.meshgrid(s1_vec, w_vec, indexing='ij')` builds the grid, and `np.argmax` together with `np.unravel_index` finds the best point. Note that there is nothing to mask out and no `np.nan` anywhere: every point of the square is a possible choice.

2. Plot utility over the nested shares **in 3D** (`ax = fig.add_subplot(1,2,1,projection='3d')` and `ax.plot_surface`) and as a **contour plot** (`ax.contourf`), with the solution marked.
3. Try $N = 50, 100, 500$ and 1000 . How much does the answer move when you add points, and how many evaluations of u does it cost? Does it improve *smoothly*?
4. Compare the two calibrations. Where is the surface steep, and where is it flat? Which of the two would you expect to be harder for an optimizer?

2.2 L-BFGS-B

L-BFGS-B is the workhorse for smooth problems with several variables and simple bounds. The **B** stands for *bounds*: it takes a box around each variable, but it does *not* take general constraints. Because we wrote the problem in nested budget shares, a box is all we need -- the whole of `solve()` is

```
minimize(self.objective, s0, method='L-BFGS-B', bounds=((0,1),(0,1)))
```

and `objective()` is just $-u$. There is nothing else to arrange.

1. Solve the problem this way and implement it as `solve()`. Compare the solution, the number of function evaluations and the running time with the grid search. Do the two methods agree?

2. Record the **convergence path** with the callback argument of `optimize.minimize`, and draw it on top of the contour plot from section 2.1. Then plot how far each iteration is from the point the solver ends at, against the iteration number, on a **log scale**. Where does the method make fast progress, and where is it slow?

Hint: `optimize.minimize` calls the function you give it as callback once per iteration, with the current point as its only argument. That is the only way to see the path: the result object holds the point the solver stopped at, and nothing about how it got there. Note that an iteration is not a function evaluation -- the solver computes u several times per iteration, for the gradient and for the line search, and the callback fires only for the steps it actually takes. It is not called for the starting point either, so put `s0` in the list yourself before you call the solver. Use `callback=lambda sk: path.append(sk.copy())`. The `.copy()` is defensive: if the solver reused one array and overwrote it, you would get the same point repeated.

3. Solve from at least six different starting points, including the middle $\left(\frac{1}{2}, \frac{1}{2}\right)$ and all four **corners** of the square. Do you always get the same answer? Does the number of iterations change?

Two corners fails in *one* of the two calibrations, and report `success = True` while doing so. Find them, and describe what happens there.

4. Vary the settings below -- they all go in `options={...}` -- and report the two shares, the utility reached and the number of function evaluations for each of them in one table:

(a) `ftol` (stop when the objective stops changing): $10^{-4}, 10^{-8}, 10^{-12}, 10^{-16}$

(b) `gtol` (stop when the gradient is small): $10^{-2}, 10^{-5}, 10^{-8}, 10^{-12}$

5. Which settings would you use? Note that you can rank them *without* knowing the true answer: we are maximizing, so a **higher utility is a better answer**. Why is a tighter tolerance not always a better answer?

3 Relative prices and demand

We now use the model. Train tickets get more expensive, and we ask what the consumer does.

1. Solve the model for p_3 from 0.5 to 3.0, holding p_1 , p_2 and I fixed, and plot the three **budget shares** and the three **quantities**. Do it for both calibrations, ideally in the same figure.
2. In which calibration does a higher p_3 **raise** the budget share of the train, and in which does it **lower** it?
3. What happens to the number of **bus** trips when train tickets get more expensive? Describe how the two calibrations differ.
4. How much does the price of a train ticket change the demand for **food**? Compare with the effect on bus trips.

4 Lump-sum taxes and product taxes

The government needs revenue, and has two kinds of instrument:

- a **lump-sum tax** T , which reduces income to $I - T$ and raises the revenue T
- **product taxes** $\tau_j \geq 0$, which raise the price of good j from p_j to $(1 + \tau_j) p_j$ and raise the revenue $\tau_j p_j x_j$

Total tax revenue is

$$R = T + \sum_{j=1}^3 \tau_j p_j x_j^*, \quad (5)$$

where x_j^* is what the consumer buys **given the taxes**. The consumer gets nothing back: the revenue leaves the model.

We compare the instruments in the simplest possible way. Two taxes that raise **the same revenue** are compared by the **utility** the consumer is left with. Higher is better.

Use `GovernmentClass` in `Government.py`. Its `set_taxes()` translates the taxes into the prices and the income the consumer faces, so everything you wrote in sections 1-3 keeps working unchanged.

1. Implement `tax_revenue()` (eq. 5).
2. Consider five product taxes: on food only, on the bus only, on the train only, the same rate on bus and train, and the same rate on all three goods. For each of them, plot the revenue as a function of the tax rate for rates from 0 to 3, one panel per calibration.
3. Which of the curves have a **top**, and which just keep rising? Find the revenue-maximizing rate and the largest possible revenue where there is a top. In particular: what is the most that a tax on train tickets alone can *ever* raise in the substitutes calibration?

Hint: Use a grid over the tax rate and pick the best point, exactly as in section 2.1. Read off your figure from question 2 how far out the grid has to go.

4. Plot the consumer's utility **against the revenue raised**, for all five product taxes and for the lump-sum tax, one panel per calibration. Which instrument is the cheapest way to raise a given revenue, and does the ranking depend on the calibration?
5. Let the government need $R = 0.20$, i.e. 2 percent of income. Use a root-finder to find the tax rate that raises exactly that for each instrument, and report the rate and the resulting utility in one table per calibration. Rank the instruments, and comment in particular on the tax on train tickets.

Hint: `optimize.root_scalar(f, bracket=[...], method='brentq')` where `f` is the revenue minus the target.

5 Extension

Extend the model in **one** direction of your own choice, solve it, and describe the results.

Note: *This is intended as an open question, and we encourage you to be creative.*